

Phase 3: the bottom bar

What was built BeamMP Race Manager Client: dhardesty99

The five buttons on the bottom bar used to say what they would cost and then do nothing. They work now. The server decides whether you may press one, holds the car still for as long as the job takes and charges the penalty. The game does the work.

1 The five buttons

Button	Hold	Penalty	What it does
Reposition	5s	60s	Puts the car back at the last checkpoint you passed, upright and facing the way it faces.
Spare tire	30s	30s	Fixes the car. Only when a tire is flat. See section 5.
Repair	60s	30s	Fixes the whole car.
Fuel +25%	20s	none	Adds a quarter of a tank.
Lights	none	none	Head lights. Works already.

Hold is how long the car is locked in place. **Penalty** is seconds added to your race time. The race clock keeps running during a hold, so a repair in a race costs 60 seconds of waiting and 30 on the clock.

Outside a run there is no hold and no penalty. The buttons just work.

Every number above lives on the server. Any of them can change without anyone reinstalling the mod, and the button on screen reads the new value.

2 What happens when you press one

1. The client checks you are in a car and stopped.
2. For Spare tire and Fuel it asks the car a question first. See section 5.
3. The server decides. It checks you are in a run, that you are not already busy, and that the button is one it knows.
4. If yes, the hold starts and the penalty is added straight away.
5. The car is frozen and a countdown appears under the race clock.
6. When the hold ends the server tells the client to do the job. The car is fixed and let go.
7. The client says how it went. A job the game could not do gives its seconds back.

The penalty is added at the start, not the end. Added at the end you could quit half way through the hold and get the repair for nothing.

3 The messages

Five, in the envelope everything else already uses.

client to server	service.use	{ which = "repair", flat = true }
client to server	service.done	{ which = "repair", ok = false, why = "no_vehicle" }

server to client	service.hold	{ which = "repair", hold = 60, penalty = 30 }
server to client	service.run	{ which = "repair" }
server to client	service.failed	{ which = "repair", why = "already_working" }

4 The server side

The server owns the decision, the hold clock and the penalty. It never touches a car. This is the middle of it.

```

local ACTIONS = {
  reposition = { hold = "reposition", penalty = "recovery" },
  spare      = { hold = "spareTire",   penalty = "flatTire", needsFlat = true },
  repair     = { hold = "repair",      penalty = "repair" },
  fuel       = { hold = "fuel",        penalty = nil },
}

function RM.service.use(pid, d)
  local which = type(d) == "table" and tostring(d.which or "") or ""
  local action = ACTIONS[which]
  if not action then return false, "no_such_action" end
  if jobs[pid] then return false, "already_working" end

  local r = RM.race.get(pid)
  local racing = r ~= nil and r.state == "running"

  if action.needsFlat and racing and not (type(d) == "table" and d.flat == true) then
    return false, "no_flat_tire"
  end

  local hold, cost = 0, nil
  if racing then
    hold = tonumber(RM.config.holds[action.hold]) or 0
    if action.penalty then
      local seconds = tonumber(RM.config.penalties[action.penalty])
      if seconds and seconds > 0 then
        RM.race.penalty(pid, seconds, action.penalty)
        cost = seconds
      end
    end
  end

  jobs[pid] = { which = which, hold = hold, endsAt = RM.now() + hold,
               cost = cost, reason = action.penalty,
               lap = r and r.currentLap or nil }
  return true, jobs[pid]
end

```

The hold clock rides the 100ms tick that already exists, so this adds no timer and no work per frame.

```

function RM.service.tick(now)
  local ready
  for pid, job in pairs(jobs) do
    if now >= job.endsAt then
      ready = ready or {}
      ready[#ready + 1] = pid
    end
  end
  if not ready then return nil end

  for i = 1, #ready do
    local pid = ready[i]
    sent[pid] = jobs[pid]
    jobs[pid] = nil
    RM.bus.queue(pid, "service.run", { which = sent[pid].which })
  end
  return ready
end

```

5 The spare tire problem, and what was done about it

BeamNG can burst a tire. It cannot mend one. The only repair it has rebuilds the whole car. So a spare tire is the same job as a repair for half the wait, and nobody would ever press Repair.

The plan offered three ways out and said the third was the default. That is what was built: Spare tire is only allowed when a tire is actually flat. It still fixes the whole car, but you cannot use it as a cheap repair because you need a flat first.

Only the car knows whether a tire is down, so the car is asked before the request goes out, not after a thirty second wait:

```
v:queueLuaCommand([[
  local flat = false
  for _, w in pairs(wheels.wheels or {}) do
    if w.isTireDeflated then flat = true break end
  end
  obj:queueGameEngineLua("extensions.raceManager_service.onFlat(" .. tostring(flat) .. ")")
]])
```

Fuel asks the same way. An electric car has no tank to fill, and being told that after twenty seconds of waiting is worse than being told now.

6 In the game

Every call was checked against the game's own files before it was written. This matters: a value invented earlier in this project stopped checkpoints working at all, and the fix for that is in this same build.

Below, `bridge` is `core_vehicleBridge`.

Call	Where the game uses it
<code>bridge.executeAction(veh, 'setFreeze', b)</code>	career, cargoScreen
<code>bridge.executeAction(veh, 'setEnergyStorageEnergy', name, v)</code>	career fuel, funstuff
<code>bridge.requestValue(veh, cb, 'energyStorage')</code>	gas stations, career fuel
<code>spawn.safeTeleport(veh, pos, rot)</code>	spawn.lua, map.lua
<code>veh:queueLuaCommand("beamstate.reset()")</code>	vehicle main.lua
<code>wheels.wheels[id].isTireDeflated</code>	beamstate, wheels, sounds
<code>obj:queueGameEngineLua(...)</code>	ai.lua, beamstate.lua

One detail worth writing down. The tanks come back wrapped: `requestValue` hands the callback a list, and the tanks are the *first item* in it, not the list itself. The game's own code reads `ret[1]`. Getting this wrong would have filled nothing and said nothing.

7 On screen

- A countdown under the race clock while the hold runs, with a bar that fills as it goes.
- The clock keeps counting during the hold, because it is still costing you time.
- The penalty shows the moment it lands.
- Every bottom bar button greys out while a hold is running.

- The label on each button is read from the server, so it always states what the server actually charges.

8 Testing

Suite	Tests	Covers
Geometry	31	course shape, grid, gate fitting
Phase 1	91	names, roles, roster, stores
Phase 2	131	racing, splits, penalties, the clock
Phase 3	50	the bottom bar
Results	103	the results screen and the race log
HUD numbers	26	the clock and the lap bar, in the browser
Total	432	

Plus 121 interface checks and 10 key bindings, both of which fail the build if the screen and the code stop agreeing.

The 50 phase 3 tests cover: no cost outside a run, the hold and the penalty inside one, the penalty landing at the start, one job at a time, the hold ending on its own and not a second early, a failed job giving its seconds back, a working job keeping them, the spare needing a flat, fuel waiting but costing nothing, an unknown button, and a hold not being left behind when a run ends or a player leaves.

The tests were then checked by breaking the code on purpose. Charging the penalty at the end instead of the start, letting the spare through without a flat, dropping the refund and dropping the cleanup were each tried in turn. Every one of them was caught. A test that passes on broken code is worth nothing, so this is the part that says the numbers above mean something.

9 What is not tested

The tests run against a mock server with no game in it. They can prove the server holds you for the right time and charges the right seconds. They cannot prove the car actually freezes, repairs, fills or moves, because that needs BeamNG running.

So this needs one pass in the game before it goes live: each button once in a race and once in free driving, and one run using every button, then read the results and check every penalty is listed with the right reason.

10 Also in this build

The recording from 29 August turned up eight bugs. Four were reported and four were not. All eight are fixed.

Bug	Cause
1 Driving through a checkpoint often did nothing.	The gate volume was turned ninety degrees. Its 20 metre width ran <i>along</i> the road and its 3 metre depth ran <i>across</i> it, so you had to be within a metre and a half of the exact line the gate was marked from.

	Bug	Cause
2	Checkpoints floated above the ground and cars passed under them.	The volume sat centred on its position while the posts were drawn upward from it. Half the box was underground and its ceiling was at four metres under a gate that looked eight tall.
3	Missing one checkpoint left the next ones stuck.	Mostly 1 and 2. There was also a real dead end: miss more than four gates and every later gate was refused for the rest of the run, which then could never finish.
4	The light menu would not close.	Nothing closed it but the button that opened it. It sat over the road and over the results screen.
5	Messages sat there with no sign they would go.	Now ten seconds, with a line across the top that fills as they run out.
6	The clock showed 4:60.0 and 6:60.0.	The minutes were taken from the raw time and the seconds rounded on their own, so 299.98 read as four minutes and 60.0 seconds. Not reported.
7	The lap bar emptied at the end of the lap.	It counted back from the next gate, and after the last gate the next one is the line again, which read as no gates done. It went from 96 percent to nothing. Not reported.
8	The clock read 3:09.4 +90 when it already included the 90.	The penalty total now sits with the cut count, where it reads as a tally and not as ninety more seconds still to come. Not reported.

Bugs 1 to 8 are on the server now. Phase 3 is built and tested but not yet deployed, because of the in game pass in section 9.

Phase 3 is code complete. The eight fixes above are live. Phase 3 goes live on your word, after one drive.